
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents are evaluated on the patch they leave behind. We show that this
2 final state hides most of what happens during a run. We built an instrument that
3 records every mutating action of an agent as a git commit on a hidden reference,
4 replays each benchmark instance’s held-out tests at every recorded state inside
5 the benchmark’s own container image, and attributes each detected regression to
6 the harness check that could have caught it. Applied to SWE-bench Verified, the
7 instrument recorded 184 regression events across the runs that produced any; the
8 final patch exposed one of them. Regression frequency depended on the model
9 stack rather than the benchmark: on the same 40 instances under the same harness,
10 one frontier stack produced no events in 40 runs and another produced 140 events in
11 11 incidents (Fisher $p = 0.010$). A pre-registered comparison of recovery policies,
12 analysed with code committed before unblinding, found that rollback to a verified
13 checkpoint left contamination in one final tree where no recovery left it in nine
14 (paired sign test, $p = 0.022$), but rollback also resolved fewer tasks, because the
15 verifier that triggers it rejected patches the official grader accepted. We release
16 the instrument, its validation protocol, and a catalogue of 28 measurement defects
17 encountered while building it, 23 of which produced plausible numbers rather than
18 errors.

19 1 Introduction

20 Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] run
21 the repository’s tests against the patch an agent submits and report whether the target tests now pass
22 and the previously passing tests still pass. Work on regressions introduced by agents follows the same
23 convention: it examines the submitted patch and counts the previously passing tests it breaks [8].

24 This convention has a blind spot. A regression that an agent introduces and later repairs leaves no
25 trace in the final patch. Any harness with a recovery mechanism (a retry, a rollback to a checkpoint,
26 or a repair step) produces exactly this pattern by design, and current evaluation cannot tell a run that
27 never broke anything from a run that broke a great deal and repaired it. Whether this matters depends
28 on two empirical questions that have not been answered: how often regressions occur during a run,
29 and how much of that activity survives to the final state.

30 We answer both by measuring the timeline rather than the endpoint. Our contributions are:

31 1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as
32 a commit on a git reference the agent cannot see. After the run, the instance’s held-out passing tests
33 are replayed at every recorded state inside the benchmark’s pinned container. Detection is attributed
34 by test coverage rather than by co-occurrence in time. The instrument’s validity is established by

positive and negative controls, a baseline liveness gate, a golden check that drives the benchmark’s own gold patch through the agent’s execution path, and a re-scoring control that re-measures archived runs under a changed instrument with no model calls (Section 3). 2. **Measurements on SWE-bench Verified.** Across the runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). Event frequency was a property of the model stack, not the benchmark: identical instances and harness gave 0 events under one frontier stack and 140 under another (Section 5.3). 3. **A pre-registered comparison of recovery policies.** With the analysis committed before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.022$) but resolved fewer tasks, and we trace the loss to verifier precision rather than to rollback itself (Sections 5.4 and 5.5). 4. **A catalogue of 28 measurement defects** found while building the instrument, classified by mechanism, with the design rules that caught most of them (Appendix A). Twenty-three of the 28 produced a plausible number rather than an error.

All measurements are exploratory and were made on a development slice of the benchmark that is excluded from any confirmatory frame. No claim is made about the population of instances beyond that slice.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by two test sets: tests that must go from failing to passing, and tests that must keep passing. Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: contamination and memorisation [6], leaked solutions and flawed tests [6, 7], and the response of rolling benchmarks that draw fresh instances [3, 5] or private repositories [4]. UTBoost [7] showed that the held-out tests themselves are often insufficient, changing the verdict on 24% of Verified leaderboard entries. TDAD [8] measured regressions in submitted patches on Verified and found them in about 30% of resolved instances. Our measurement is complementary to TDAD’s: we measure the same quantity on the timeline rather than the endpoint, and Section 5.2 reports how the two compare on the same runs.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], and Agentless [11] established the design space of coding-agent scaffolds; mini-swe-agent [12] reduced it to a bash-only loop. CodeAct [13] and ReAct [14] are the underlying action formalisms. Within-episode recovery has been studied as verbal self-reflection [15, 16], progress-gated recovery [17], checkpoint repair for program-of-thought [18], and provenance-based rollback [19]. Operating-system framings for agents [20] treat checkpointing and scheduling as shared primitives. Process reward models and rubric verifiers for software agents [24, 25] address the verifier-precision problem we encounter in Section 5.5 from the training side.

Evaluation infrastructure. Trajectory-level diagnostics [22] and holistic leaderboards [21] argue that resolve rate alone is uninformative; long-horizon benchmarks [23] make the same point with harder tasks. Our instrument is a specific, validated trajectory-level measure of one quantity.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of automated program repair [26, 28, 29]. A study of two million patches [27] showed that regression tests are the primary defence against overfitting patches; our results concern what happens to regressions that the agent’s own process repairs before the patch is ever tested.

3 Instrument

Observational timeline. The harness wraps every mutating tool call. After each call, the working tree is committed to a git reference outside the agent’s view, using a private index so that the agent’s own git status and git diff are unchanged. Rollbacks and the end of the run are recorded as observations too. Because observations are commits, an archived run can be re-measured later without re-running the agent.

Exhaustive replay. For each observation, the instance’s passing-test set is run inside the benchmark’s pinned container against that observation’s tree. A regression event is a test that passes at one observation and fails at a later one. We replay every observation rather than bisecting between the start and the end, because bisection assumes the verdict sequence is monotone, and a recovery policy violates that assumption by construction. Infrastructure failures during replay (a container that

will not start, a patch that will not apply, a timeout, a log with no results) are recorded as missing observations, never as failures. Baseline-dead tests, which fail at the first observation, are excluded from event counting.

Attribution. A regression is attributed to a harness check if a failing check and the broken held-out test both exercise a file the agent changed at that observation, using a per-instance coverage map built at the base commit. We report attributed detection, co-occurrence (any harness failure while the regression was open, an over-count), and unknown separately.

Execution environment. The agent’s tools and the harness’s verification checks execute inside the same pinned container as the replay, with file changes synchronised between the container and the host tree that the timeline records. Section A.2 describes why this was not initially the case and what it cost.

Validation. Five gates run before any paid experiment: a negative control (clean runs must measure zero), a positive control with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline liveness check that the probes actually return passes. A golden check drives the benchmark’s gold patch through the agent’s real tool path and requires the official grader to return "resolved" (and a null run to return "unresolved"); it passed on the three repository families with the most distinctive test runners. A re-scoring control re-measures an archived run’s committed timeline under a changed instrument with no model calls. It was run twice on different archived runs and reproduced their episode counts exactly both times.

4 Experimental setup

Benchmark and slice. SWE-bench Verified, 500 instances. A 40-instance development slice, drawn by stratified sampling and fixed before any measurement, was used for all experiments here; it is excluded from the confirmatory frame that a later study would use.

Harness. A planner decomposes the task into steps, each with a verification command; a worker executes each step with three tools (read file, write file, run shell); a monitor runs the verification. On failure, the recovery policy acts. Three policies were compared: **rollback** (reset to the last verified checkpoint and retry with feedback), **repair-in-place** (retry from the failed tree), and **no recovery** (keep the failed step’s tree and continue). Each run had a \$4 work-cost cap; budget exhaustion is scored as failure.

Models. Two frontier stacks were run under identical harness, policy, instances, and caps: Claude (Opus 4.7 planner, Sonnet 4.6 worker) and GPT-5.6 (sol planner, terra worker). Model identifiers are recorded in every run’s manifest.

Outcomes. Resolve is the official grader’s verdict on the final patch. Event counts are reported at three levels because the distribution is heavily overdispersed: distinct incidents (observations at which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs (runs with at least one event). Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the recovery-policy contrast were declared before the relevant data existed. The contrast’s analysis (exact paired sign tests on final-state contamination as primary and on incident exposure as co-primary) was committed to the repository before either comparison arm finished. One amendment, making the gates conditional on the model stack, was filed after the cross-stack result and before the contrast was unblinded. Section B gives the wording.

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) at \$34.24; the GPT-5.6 stack resolved 13 of 35 graded instances (37.1%) at \$8.14. The GPT-5.6 sweep’s circuit breaker stopped three cells after six consecutive zero-progress failures in one repository family, and two cells failed on a file-synchronisation defect; both are counted as attempted and not resolved.

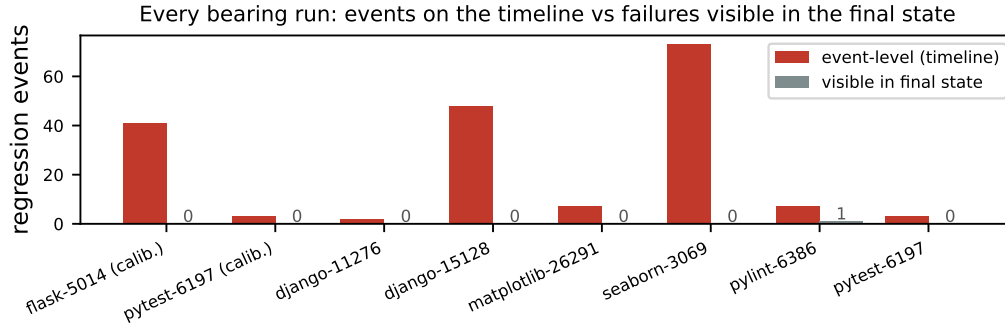


Figure 1: Every run that produced at least one regression event, in both GPT-5.6 sweeps. Red bars are events recorded on the timeline; grey bars are held-out test failures visible in the graded final patch.

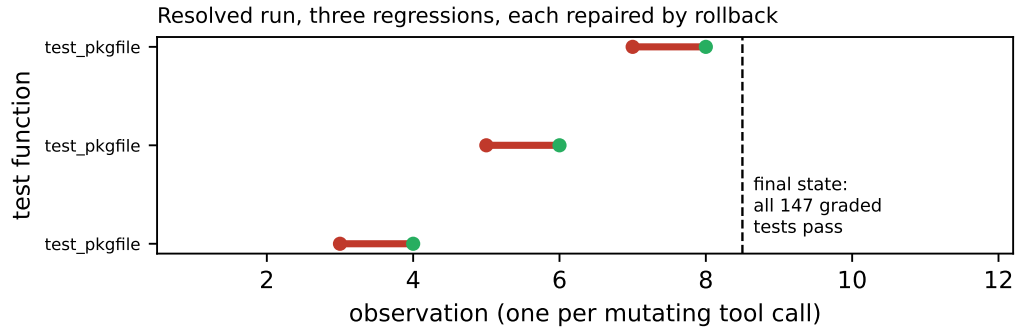


Figure 2: One run that the official grader marked resolved, with all 147 graded tests passing. The timeline shows three regressions of the same test function, each repaired by rollback two observations later.

135 5.2 The final state hides almost all regression activity

136 Figure 1 shows every run that produced a regression event, under the rollback policy with the GPT-5.6
 137 stack. The timeline recorded 184 declared events across eight runs. The final patch exposed one.
 138 Every other event was created and then removed by the harness’s rollback before the run ended, so
 139 the graded patch was clean. The same measurement made from the final patch, which is how prior
 140 work measures regressions [8], would report one event in eight runs.

141 Figure 2 shows a single run in detail. The grader marked it resolved with all graded tests passing. Its
 142 timeline contains three regressions of the same test function at observations 3, 5, and 7, each repaired
 143 by rollback. Two of the three were not detected by any harness check while they were open; the
 144 harness noticed nothing and the rollback that repaired them was triggered by an unrelated failure.

145 Across the full GPT-5.6 sweep, 48 of 162 raw episodes had no co-occurring harness failure of any
 146 kind.

147 5.3 Regression frequency depends on the model stack, not the benchmark

148 On the same 40 instances, under the same harness and rollback policy, the Claude stack produced
 149 no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140
 150 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2%
 151 to 32.0%). A one-sided Fisher exact test on the bearing fractions gives $p = 0.010$. Observation density
 152 differs by a factor of 1.4 between the stacks, which does not account for a difference of 140 to 0.

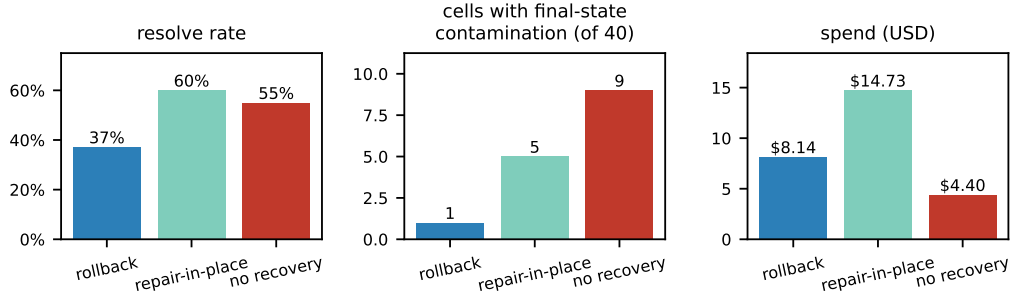


Figure 3: The three recovery policies under the GPT-5.6 stack on the same 40 instances.

The Claude zero is a measured zero. The same stack’s earlier two-instance canary run did produce three events, and re-scoring that run’s archived timeline under the instrument used for the 40-run sweep reproduces them exactly, so detection did not fail; these 40 runs did not break adjacent behaviour.

We initially interpreted a low event rate as a property of the benchmark, and a pre-declared rule directed us to switch to a benchmark with a larger held-out test set. The second stack falsified that interpretation on the same instances. Regression frequency is a property of the agent regime, the combination of model stack and recovery policy, and any gate on it must be declared per regime.

5.4 Recovery policy: rollback keeps the final tree clean

Figure 3 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on 25 (exact sign test, $p = 0.022$). Repair-in-place was worse on 5, better on 1, and tied on 29 ($p = 0.22$). The co-primary endpoint, incident exposure, did not differ between policies ($p = 0.45$): regressions occur under every policy at similar rates, and the policy determines whether they persist.

One no-recovery run illustrates the mechanism. Its final tree contains the literal string `$(cat django/db/models/expressions.py)` as the first line of the module, a shell idiom the model pasted into a file write. Every one of the instance’s 137 graded tests failed to run. Under rollback the same model’s failed attempt on the same instance was reset, and the run’s final patch passed all 137.

Disclosure. The first unblinding of this comparison gave $p = 0.375$ for the primary endpoint. Seven cells (five no-recovery, two repair-in-place) had been dropped as ungradable because their final trees made the test suite uncollectable and the grader returned no result. The official grading rule scores such a patch as failing every test. These were the most severely contaminated final states in the comparison, and they had been removed from the endpoint they bore on most. The grader was corrected to distinguish an environment failure (no results with or without the patch) from a patch that kills the suite (results at baseline, none with the patch), and the seven archived trees were re-graded without re-running any agent. All statistics above use the corrected verdicts.

5.5 Rollback loses resolution to verifier precision

Rollback resolved fewer tasks than either alternative (Figure 3, left): 37% against 60% for repair-in-place and 55% for no recovery. Paired by instance, rollback won one discordant pair and lost nine against repair-in-place (McNemar exact $p = 0.022$), and won three and lost nine against no recovery ($p = 0.15$).

The loss is attributable to the verifier. Of the 18 rollback runs that failed, 15 failed at the first step, and 9 of the 18 instances were resolved under a policy that kept the rejected work. In those cases the monitor’s check, a shell command written by the planner, rejected a patch that the official grader accepted, and rollback discarded it. Rollback’s advantage on the primary endpoint and its disadvantage on resolution therefore have the same cause: it acts on the verifier’s verdict, and the verifier’s precision sets the exchange rate between a clean final tree and a solved task.

190 This suggests a specific improvement, which we have not yet evaluated: replace the planner-written
191 check with the repository’s own tests that cover the files a step touched, selected by the same coverage
192 map the instrument uses for attribution, and trigger rollback only on a genuine regression. The
193 instrument’s observation layer would then serve as the harness’s monitor.

194 6 Discussion

195 **Implications for agent-OS design.** If checkpointing and recovery become shared primitives, the
196 results here argue that timeline observability must be one too. A recovery primitive without an
197 observation primitive makes runs look cleaner while hiding the activity that distinguishes policies.
198 The 184-to-1 ratio in Section 5.2 is not a property of one harness; it is what any final-state grader will
199 report for any harness that repairs its own mistakes.

200 **On the measurement defects.** Building the instrument produced 28 defects, catalogued in Appendix
201 A. Twenty-three produced a plausible number rather than an error, and 27 were present while a passing
202 test suite watched. They fall into four mechanisms: dependence on ambient host configuration,
203 infrastructure failure rendered as a measurement, final-state measurement of an event-shaped quantity,
204 and consuming a producer’s output without checking that the producer succeeded. Each mechanism
205 has public instances in benchmark infrastructure (Appendix A.3). The two rules that prevented most
206 of them are cheap: type infrastructure failure as a missing observation, never as a test verdict; and
207 require the instrument to prove it is alive, not merely that it did not report anything.

208 7 Limitations

209 All measurements come from a 40-instance development slice, one sweep per arm and per stack.
210 Run-to-run variability within a regime is unmeasured; the confirmatory design specifies seeded
211 replications on held-out instances. The cross-stack comparison cannot separate the model from its
212 provider adapter, which is part of the regime. The held-out tests observe roughly the blast radius of
213 the gold test patch, so "regression" here means a regression adjacent to the edit, and event counts
214 are lower bounds. Confidence intervals treat the fixed slice as the population and ignore repository
215 clustering (46% of instances are from one repository). The harness is our own; the instrument has not
216 yet been applied to a public scaffold.

217 8 Conclusion

218 Final-state evaluation of coding agents is blind to regressions that the agent’s own process repairs, and
219 in our measurements that is nearly all of them. Measuring the timeline is feasible, validates against
220 controls, and separates recovery policies that the final patch cannot distinguish. It also exposes the
221 cost of recovery: rollback keeps the final tree clean only as reliably as its verifier is precise.

222 References

- 223 [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR
224 2024. arXiv:2310.06770.
- 225 [2] OpenAI. Introducing SWE-bench Verified. 2024.
- 226 [3] L. Zhang et al. SWE-bench Goes Live! arXiv:2505.23419, 2025.
- 227 [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?
228 arXiv:2509.16941, 2025.
- 229 [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontami-
230 nated evaluation of software engineering agents. arXiv:2505.20411, 2025.
- 231 [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of
232 reason. arXiv:2506.12286, 2025.
- 233 [7] H. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025.
234 arXiv:2506.09289.

235 [8] TDAD: Regressions introduced by coding agents on SWE-bench Verified. arXiv:2603.17973,
236 2026.

237 [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering.
238 NeurIPS 2024. arXiv:2405.15793.

239 [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents.
240 ICLR 2025. arXiv:2407.16741.

241 [11] C. Xia et al. Agentless: Demystifying LLM-based software engineering agents.
242 arXiv:2407.01489, 2024.

243 [12] SWE-agent team. mini-swe-agent. Software, 2025.

244 [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.

245 [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023.
246 arXiv:2210.03629.

247 [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.
248 arXiv:2303.11366.

249 [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023.
250 arXiv:2303.17651.

251 [17] ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process
252 routing. arXiv:2511.14584, 2025.

253 [18] REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.

254 [19] From agent traces to trust: Evidence tracing and execution provenance in LLM agents.
255 arXiv:2606.04990, 2026.

256 [20] K. Mei et al. AIOS: LLM agent operating system. arXiv:2403.16971, 2024.

257 [21] Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation.
258 arXiv:2510.11977, 2025.

259 [22] What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184,
260 2026.

261 [23] SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios.
262 arXiv:2512.18470, 2025.

263 [24] Agentic rubrics as contextual verifiers for SWE agents. arXiv:2601.04171, 2026.

264 [25] SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.

265 [26] Patch overfitting in program repair: A survey. 2024.

266 [27] Y. Wang et al. When automated program repair meets regression testing: An extensive study on
267 two million patches. ACM TOSEM, 2024. arXiv:2105.07311.

268 [28] Patch correctness assessment: A survey. ACM TOSEM, 2024. doi:10.1145/3702972.

269 [29] H. Ye et al. Automated patch assessment for program repair at scale. Empirical Software
270 Engineering, 2021.

271 [30] SWE-bench issue #601: Test result hijacking via stdout forging in the evaluation harness.
272 github.com/SWE-bench/SWE-bench, 2025.

273 [31] OpenHands issues #4235 and #7044: Environment errors in SWE-bench instance evaluation.
274 github.com/OpenHands/OpenHands.

275 [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. arXiv:2504.21798,
276 2025.

277 [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym.
278 arXiv:2412.21139, 2024.

279 [34] Context as a tool: Context management for long-horizon SWE agents. arXiv:2512.22087, 2025.

A The measurement defect catalogue

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure a state where an event was defined; class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine’s git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare <code>python</code>	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare <code>pytest</code> from <code>PATH</code>	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare <code>python</code>	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on <code>stderr</code> , markers on <code>stdout</code>	one framework’s results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container <code>workdir</code> unvalidated	every <code>exec</code> exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family’s oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host. Four further defects found after this census closed are described where they arose (Section 5.4 and the repository log).

A.2 The defect that invalidated a conclusion

After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate. We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source checkout, while every probe and every gate ran inside the pinned container. On the host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace package, and everything downstream fails in ways indistinguishable from an incompetent agent. Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and never the agent’s execution path. The fix was to route the agent’s tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

303 **A.3 Mechanisms in public infrastructure**

304 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
305 bench images. Class B: UTBoost’s finding that the held-out oracle is insufficient at leaderboard scale
306 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
307 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
308 Class D: the SWE-bench grader accepting forged test output on stdout [30].

309 **B Pre-declaration and analysis**

310 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
311 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
312 of at least 25%; after the cross-stack result they were re-declared per (substrate, model stack) before
313 the contrast was unblinded. The contrast’s primary endpoint is final-state contamination, paired by
314 instance, exact two-sided sign test; the co-primary is incident exposure. The analysis script was
315 committed before either comparison arm finished. Budget exhaustion is scored as failure. Instances
316 whose baseline oracle records failures in the unmodified image are excluded from resolve-rate
317 comparability claims, and their dead tests are typed as missing observations for event counting.